Temat 5: Automatyczna klasyfikacja odpadów (segregacja ze zdjęcia)

Typ zadania: klasyfikacja obrazów (niezbalansowane dane, mały zbiór)
Dataset: TrashNet — 2 527 zdjęć, 6 klas (glass 501 / paper 594 / cardboard 403 / plastic 482 / metal 410 / trash 137), zdjęcia 512×384 na jednolitym tle

Opis problemu

Błąd klasyfikatora w sortowni kosztuje: plastik sklasyfikowany jako papier zanieczyszcza partię surowca. TrashNet jest celowo mały (2 527 zdjęć) i niezbalansowany — to nie wada, tylko realny warunek: tak właśnie wygląda dane, które sam ręcznie zebrałeś, zamiast milionów zdjęć z internetu.

Zadanie mały zbiór ma własne reguły: pojedynczy podział train/test daje zbyt losowe wyniki (różnice kilku punktów procentowych między seedami), więc wymagana jest walidacja krzyżowa (k-fold) lub wielokrotne powtórzenie podziału z różnymi seedami — żeby wynik był liczbowo wiarygodny. W analizie błędów: plastik przezroczysty vs szkło, papier vs karton — sprawdźcie, dlaczego model myli te pary i czy dadzą się naprawić.

Dataset

Zbiór: **TrashNet**. Dane: 2 527 zdjęć, 6 klas (glass 501 / paper 594 / cardboard 403 / plastic 482 / metal 410 / trash 137), zdjęcia 512×384 na jednolitym tle.

Pobieranie:

- Kaggle (dataset-resized, 41 MB): https://www.kaggle.com/datasets/feyzazkefe/trashnet
- Repo GitHub (oryginalne dane + link do wersji pełno-res): https://github.com/garythung/trashnet

Baseline i wymagane warianty

Baseline: ResNet-18 z wagami ImageNet na 224×224

Co najmniej **dwa warianty / modele** do porównania (na tym samym podziale danych):

- model trenowany od zera (celowo: pokazać koszt małych danych)
- transfer + k-fold cross-validation (5-fold)
- Oczywiście można zaproponować inne.

Wymagane metryki

- accuracy
- macro-F1 (wiodącą: niezbalansowany zbiór)
- confusion matrix + recall per klasa

Eksperyment

wpływ augmentacji i liczby danych: pełny zbiór vs 50% vs 25% — jak mocno augmentacja zastępuje potrzebną liczbę zdjęć?

Specyfikacja szczegółowa (twarda)

- **Podział danych:** 70/15/15 (train / walidacja / test)
- **Seed treningu:** 6102 — stałe ziarno losowości (numpy i torch), identyczne dla obu wariantów.
- **Próg metryki głównej:** 0,80 — osiągnięcie progu **lub** udokumentowany w raporcie wniosek, dlaczego próg nie jest osiągalny (co go blokuje).
- **Wymagany element własnej implementacji:** własnoręcznie zaimplementowana pętla k-fold cross-validation (bez sklearn.KFold).

- Baseline trenowany **od zera** (chyba że temat stanowi inaczej); zakaz wytrenowanych modeli z Kaggle / repo — dozwolone tylko wagi wskazane w temacie (standardowo ImageNet); dane mogą pochodzić z Kaggle / HF.
- Budżet obliczeniowy na cały projekt: **do 2 h na GPU lub 4 h na CPU**.

Wymagania projektu

- **Dataset + analiza danych.** Wczytanie zbioru, podstawowa analiza: liczba przykładów, rozkład klas, rozmiary obrazów, wizualizacja przykładów (siatka klas).
- **Minimum dwa modele / dwa warianty** (jeden = baseline, drugi — własny wybór) porównane na tym samym podziale danych.
- **Eksperyment** (architektura / augmentacja / liczba danych / hiperparametry) z raportem, co zmienia i dlaczego — nie tylko tabela wyników.
- **Minimum 2–3 metryki** adekwatne do zadania; dla niezbalansowanych danych sama accuracy jest niedostateczna.
- **Analiza błędów**: confusion matrix, przykłady błędnych predykcji, omówienie przyczyn (dane? architektura? augmentacja? etykiety?).
- **Wizualizacje**: wykresy treningowe, confusion matrix, przykładowe predykcje — zapisane w repo (nie tylko w raporcie).
- **Wandb (Weights & Biases)**: eksperymenty logowane do publicznego dashboardu; link do run-u w README. Demo: notebook `.ipynb` (nie Gradio/Streamlit).
- **README + reprodukowalny kod.** Uruchamianie krok po kroku na GPU *lub* CPU; w repo bez `__pycache__/`, wag `.pt/.pth`, surowych danych — tylko kod, wykresy i instrukcja.
- **Wnioski**, nie tylko accuracy: co działało i czemu, ograniczenia, co robilibyście inaczej, odniesienie do praktycznego zastosowania.

Zasady oddania

- Repozytorium kodu przesyłane przez Moodle — **każda osoba w zespole wysyła własne przesyłanie**.
- Termin: **do dnia ostatnich zajęć**; prezentacja odbywa się **stacjonarnie na ostatnich zajęciach**.
- Repo nie może zawierać: `__pycache__/`, wag `.pt/.pth`, surowych danych.
- Dokumentacja projektu zawiera skład zespołu: **imię i nazwisko, nr albumu, e-mail** dla każdej osoby.

Regulamin użycia narzędzi AI

Pomoc AI jest dopuszczalna tylko w poniższych zakresach — każde użycie wymaga wpisu w `AI-UZYCIE.md`: **model** (np. GPT-5, Claude Sonnet) + **pełny prompt** (skopiowany lub w pliku) + zakres (opis / debugowanie):

- interpretowanie błędów i debugowanie własnego kodu;
- wyjaśnianie konceptów teoretycznych (do własnego zrozumienia);
- poprawa stylu opisów (nie zmiana merytoryki).

Zakazane: generowanie kodu przez AI (nawet krótkich fragmentów — kod musi powstać samodzielnie), wygenerowanie całości eksperymentu lub raportu przez AI, kopiowanie kodu z tutoriali bez rozumienia, gotowe pipeline'y pokrywające cały etap, brak deklaracji. **Naruszenie regulaminu: 0 pkt za projekt.**

Deklaracja w `AI-UZYCIE.md` jest obowiązkowa także wtedy, gdy AI nie było używane — wchodzisz i piszesz: „narzędzia AI nie były używane".

Oświadczenie zespołu w dokumentacji: „Praca wykonana samodzielnie; użycie AI (lub jego brak) jest deklarowane w pliku `AI-UZYCIE.md`. Uczestnicy: [imię, nazwisko, nr albumu, e-mail]."
